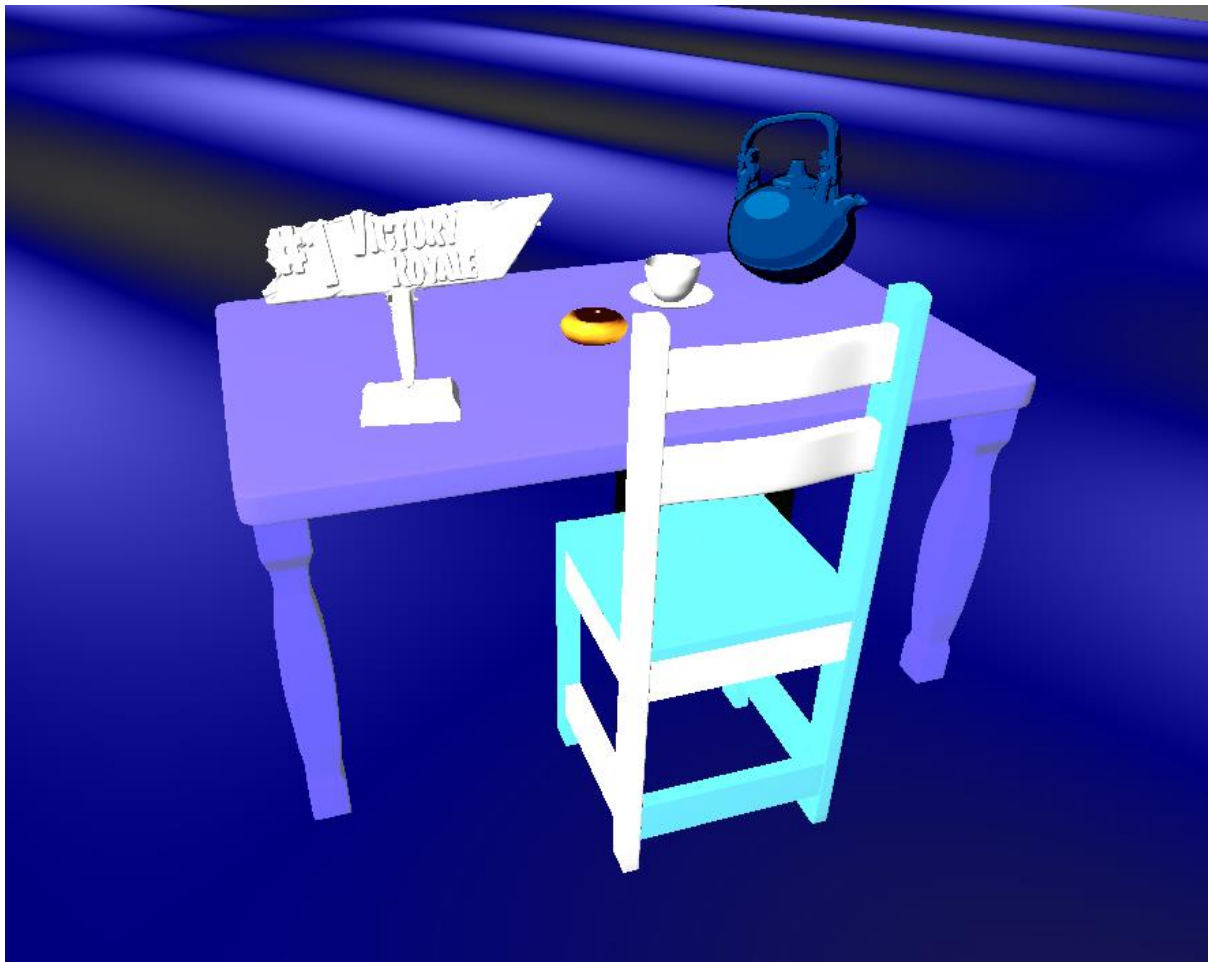


## LAB 3

Per il **lab 3** il tema della scena è il tè del pomeriggio. Ho infatti inserito una serie di mesh scaricate online. Ora nella scena c'è un tavolo e su di esso sono stati appoggiati una tazza e una teiera. Inoltre, ho tenuto anche il toroide che ora è ruotato di 90 gradi e rappresenta una ciambella. Inoltre sotto al tavolo è presente un cubo.



*Figura 1 - Scena completa*

### 1 Oggetto scansionato

L'oggetto che ho scansionato in laboratorio è un soprammobile che rappresenta il messaggio di vittoria di un popolare videogioco. Una volta effettuata la scansione con lo **Shining 3d Einstar**, ho elaborato la nuvola di punti con il software **EXtar** con il quale ho ottenuto una

mesh. Per il post processing ho prima eliminato tutte le floating parts, chiuso i fori per ottenere una mesh completa e infine ho semplificato ed ottimizzato il modello per ridurre le dimensioni del file .obj, così da renderlo più leggero e facilmente gestibile.



Figura 2 - Oggetto scansionato

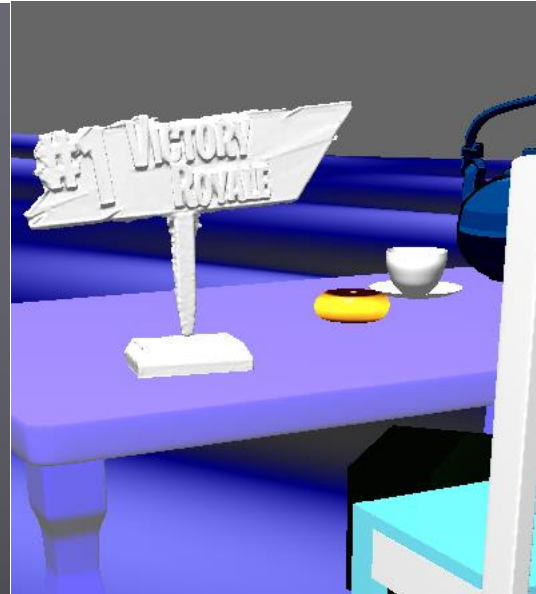
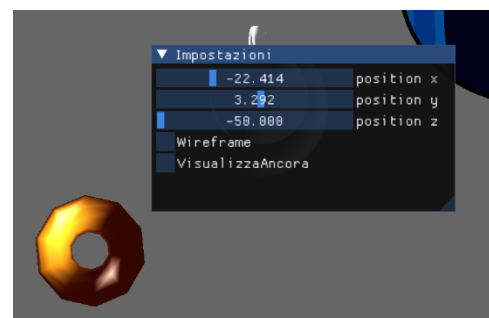
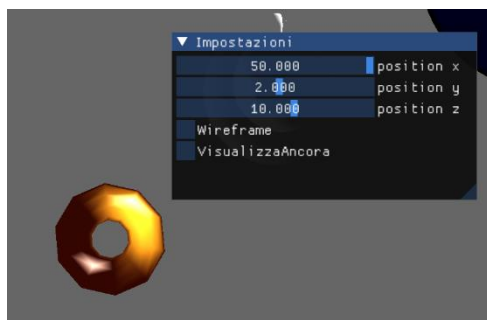


Figura 3 - Scansione inserita nella scena

## 2. Phong

Per ottenere la resa degli oggetti con shader **Phong** ho modificato il vertex e il fragment shader. Per prima cosa nel vertex vado a definire *gl\_position* che moltiplica le coordinate del vertice per le matrici di modello, di vista e di proiezione. Poi calcolo una serie di dati che mi serviranno nel fragment cioè:



*vcsPosition* (posizione del vertice nello spazio telecamera), *vcsLightPosition* (posizione della luce nello spazio camera) e *vcsN* (normale al vertice, è necessario usare transpose e inverse perché, se moltiplicassimo semplicemente per la matrice perderemmo l'ortogonalità). Nel fragment shader ho quindi calcolato la luce ambientale, quella diffusa, quella speculare e poi le combiniamo. Nella scena Phong è applicato al toroide.

### 3. Shader di movimento nel tempo

Ho esteso vertex e fragment shader per realizzare un'animazione di tipo **wave motion**. Nel vertex shader ho modificato la coordinata y dei vertici della mesh in base alla formula:  $offset = a * \sin(\omega * t + 10 * vx) * \sin(\omega * t + 10 * vz)$ , dove  $a = 1$  è l'ampiezza e  $\omega = 1$  è la frequenza, per simulare un'onda che si propaga nel tempo. Il parametro t (time) è passato dall'applicazione. Il parametro offset viene passato al fragment shader per calcolare la variazione di colore.

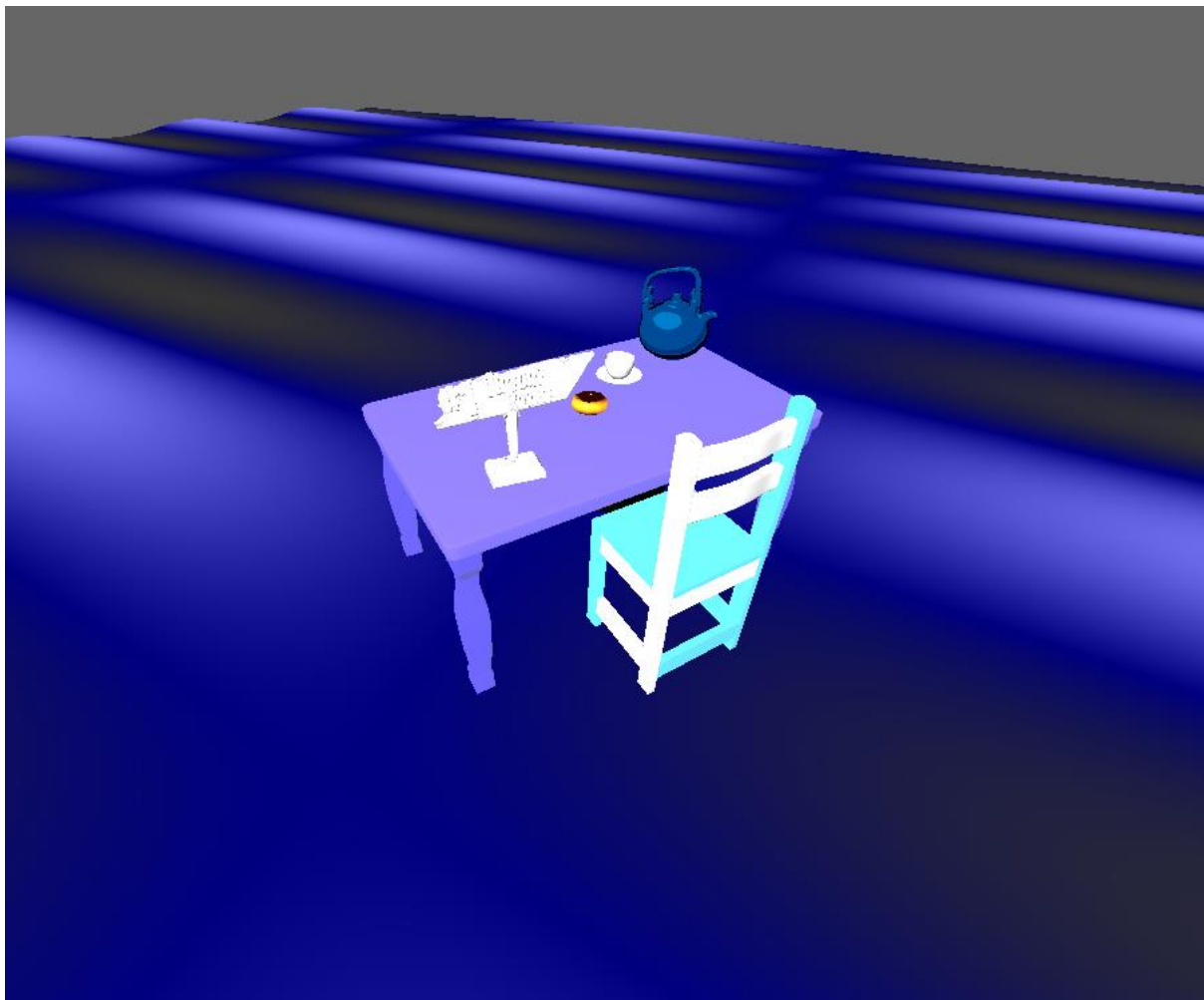


Figura 4 - Wawe

## 4. Picking

Il **picking** permette di selezionare un oggetto mediante il click della rotellina del mouse sull'**ancora** associata ad uno specifico oggetto. Nella nostra scena abbiamo un'ancora per ogni oggetto e su ognuna delle ancore è stata costruita una sfera virtuale che utilizzeremo per calcolare l'intersezione con il raggio generato dal click del mouse. Quando l'utente clicca lo fa su un pixel della viewport. Dopo il click sarà necessario calcolare il raggio che parte dall'origine delle coordinate **VCS** (camera space) e che passa per il punto **P** dove è stato effettuato il click. Le coordinate pixel di **P** sono ottenute dalla funzione *glfwGetCursorPos()* e verranno usate nella funzione *get\_ray\_from\_mouse()* per ottenere il raggio. Per ottenere questo raggio, le coordinate del click in pixel vengono prima trasformate nello spazio **NDC** (Normalized Device Coordinates), in cui *x* e *y* sono mappati in  $[-1, 1]$  e a cui si assegna  $z = -1$  per rappresentare un punto sul **near plane**. Successivamente, questo punto NDC viene trasformato nello **spazio mondo (WCS)** applicando le matrici inverse di proiezione e vista ( $P^{-1}$  e  $V^{-1}$ ). Otteniamo così un punto nello spazio attraverso cui far passare il raggio. Il raggio risultante ha origine nel centro della camera (*o*) e direzione calcolata come  $(P - o)$ . Il raggio avrà equazione:  $P(t) = o + t * (P - o)$ . A questo punto ad ogni click della rotellina del mouse dovremmo controllare tutti gli oggetti e vedere se il raggio interseca la sfera costruita sull'ancora assegnata ad ogni oggetto e prendere l'intersezione con *t\_dist* minore. La sfera per gli oggetti .obj ha un raggio maggiore rispetto agli elementi 3d generati dal codice come ad esempio il toro.

```
Oggetto Selezionato: fortnite.obj  
Operazione:  ????
```

```
Oggetto Selezionato: pot.obj  
Operazione:  ????
```

```
Oggetto Selezionato: table.obj  
Operazione:  ????
```

## 5. Traslare/Ruotare/Scalare

Gli oggetti possono essere traslati, ruotati o scalati una volta che sono stati selezionati tramite il picking. Nell'immagine sottostante il tavolo è stato ruotato e la teiera è stata ingrandita e traslata.

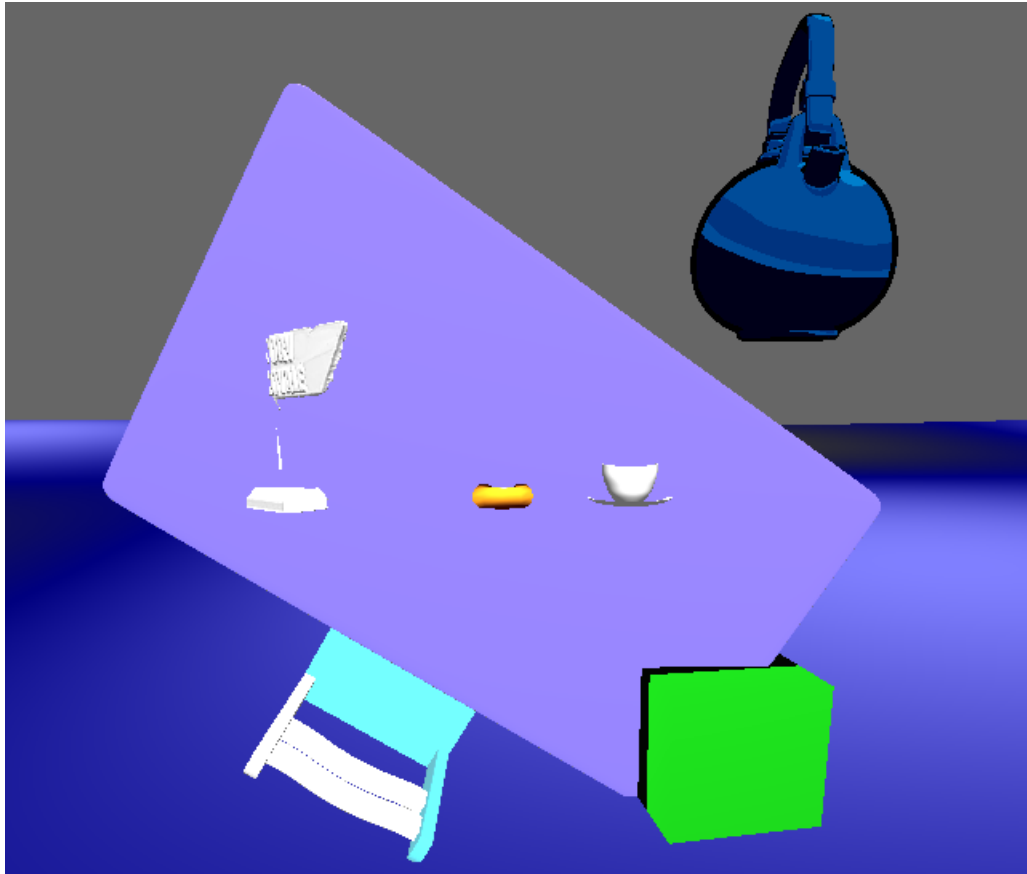


Figura 5 - Oggetti ruotati/traslati/scalati

## 6. Toon Shading

Per lo shader che applica uno stile originale ho realizzato un **toon shading** che restituisce un aspetto stilizzato agli oggetti della scena. Lo stile usa bande di colore invece di sfumature continue. L'intensità della luce, calcolata nello shader come componente diffusa del modello di illuminazione Phong, viene quantizzata in fasce discrete. In base al valore dell'intensità, viene assegnato uno specifico colore



Figura 6 - Toon shading

uniforme, evitando gradazioni intermedie e creando così un effetto di **illuminazione piatta**.

## 7. Navigazione

Per realizzare la navigazione ho inserito una funzione (*initBezierCurve*) che permette di inizializzare una curva di **Bézier** i cui punti di controllo sono fissi. I punti sono disposti in modo da approssimare una **circonferenza chiusa** così da ottenere un percorso ciclico. Durante il ciclo di rendering, se viene premuto il tasto 'B', la variabile *moveCameraOnBezier* viene impostata a true, attivando l'animazione della telecamera lungo il percorso definito dalla curva. Premendo invece il tasto 'N', l'animazione si interrompe. Nel ciclo principale (*while (!glfwWindowShouldClose(window))*), se *moveCameraOnBezier* è attiva, la posizione della telecamera viene aggiornata ad ogni frame, seguendo i punti precalcolati sulla curva (ottenuti con l'algoritmo di De Casteljau). La camera guarda costantemente verso un punto fisso nello spazio, ottenendo così un'animazione di tipo **COI** (*Camera On Interest*). L'indice *currentPoint* viene aggiornato ad ogni frame e riportato a zero una volta raggiunto il termine, in modo da creare un loop continuo.